DeepSanitizer: Combining Heuristic Rules and Deep Learning Models to Spot Silent Buffer Overflows in Binary

Zhilong Wang, Chen Cao, Li Yu, Suhang Wang, Peng Liu, Xiaoyan Sun, Anoop Singhal

Abstract—Memory sanitizer is a widely employed technique that serves as the frontline in the detection and analysis of memory corruption bugs, such as buffer overflows. Despite its prevalence, existing binary-level sanitizers face challenges in effectively identifying silent buffer overflows – instances where only one or a few adjacent variables are corrupted and usually obvious "symptoms" such crashes are not manifested – due to the complexities of inferring buffer/array boundaries in the absence of source code.
This paper introduces *DeepSanitizer*, a deep-learning-assisted memory sanitizer designed to spot silent buffer overflows. Departing from traditional techniques that attempt to infer data object boundaries, *DeepSanitizer* relies on abnormal data dependencies and memory access patterns in the overflowed region for buffer overflow detection. By combining heuristic rules and deep learning models, it discerns the distinctions between normal and out-of-bound buffer accesses, predicting whether a buffer segment contains overflows. We implemented a prototype on the Linux system and conducted a series of experiments to assess its effectiveness and performance. Results from our evaluation demonstrate that *DeepSanitizer* achieves a high accuracy in detecting silent buffer overflows. Additionally, we explored its efficacy in identifying real-world vulnerabilities through experimentation.

Index Terms—Sanitizer, Buffer Overflow, Vulnerability Analysis, Deep Learning.

1 Introduction

A fundamental challenge in cybersecurity is that vulnerabilities widely exist in all kinds of programs despite that software engineers and security analysts have been spending lots of effort avoiding and testing them [1]. Among all types of vulnerabilities, memory corruption attacks are arguably the most dangerous one, as they allow the adversary to hijack a program's control flow by modifying the return address and the nearby variables, respectively. Among all the nearby variables that can be modified, for decades, code pointers or data pointers have been preferred by attackers, because these pointers provide a most convenient way for attackers to hijack the control flow. However, pointer-modifying memory corruptions are likely to be noticed during software fuzz testing because they often manifest visible "symptoms" (e.g., crashes, failed assertions). Moreover, as control-flow protection mechanisms get mature [2, 3, 4, 5, 6] and widely deployed [7, 8, 9, 10], it becomes difficult for attackers to corrupt control data, like return addresses or function pointers, and launch control-flow hijacking attacks [11, 12, 13, 14].

The difficulty (to corrupt control data) has been motivating attackers to search for novel evasive hacking vectors, and they have recently found potentials from data-oriented attacks [15, 16, 17, 18, 19, 20, 21]. In a data-oriented attack, attackers leverage memory corruption vulnerabilities to modify some *non-control, security-critical* data. The corrupted data enables the attacker to attain escalated privileges, authentication bypass, and/or additional permissions, without being thwarted by control-flow protection mechanisms. Since data-oriented attacks do not require modification of any pointers, even if a buffer does not have any pointer variables stored in its adjacent higher-address memory locations, the buffer could be overflowed to launch serious data-oriented attacks. Moreover, when this kind of buffer overflows happen, visible "symptoms" such as crashes are usually not manifested.

This intrinsic characteristic of data-oriented attacks makes it more challenging for the existing *memory sanitizers* to do a satisfactory job. A memory sanitizer is essentially a memory error detector. All fuzz testing tools rely on memory sanitizers to tell whether an input has triggered a memory corruption vulnerability. Therefore, if a memory sanitizer is unable to identify an important category of memory errors, the fuzz testing tool that employs the sanitizer would become "blinded" when the corresponding buffer overflows are actually happening right in front of "eyes" of the fuzz testing tool.

Therefore, in this work, we seek to understand and address a major limitation of the state-of-the-art memory sanitizers caused by data-oriented attacks: existing memory sanitizers cannot detect *silent* buffer overflow memory errors when source code is *not* available. By "silent", we mean the buffer overflow incidents that don't manifest any visible "symptoms" (e.g., crashes, failed assertions). For example, when overflow incidents happen to vulnerable buffers that don't have any adjacent pointer variables stored in the memory, the memory errors usually stay silent.

Whether source code is available or not can be a critical factor affecting the adopted approach and the actual performance of memory sanitizers, especially considering that many commercial off-the-shelf (COTS) software products are binary-only. When the source code is available, Ad-

• *Zhilong Wang, Chen Cao, Li Yu, Suhang Wang, and Peng Liu are with The Pennsylvania State University, State College, PA, 16802. Xiaoyan Sun is with Worcester Polytechnic Institute, Worcester, MA 01609. Anoop Singhal is with National Institute of Standards and Technology, Gaithersburg, MD 20899. E-mail: zzw169@psu.edu; ccao@psu.edu; liyupsu@gmail.com; szw494@psu.edu; pxl20@psu.edu; xsun7@wpi.edu; anoop.singhal@nist.gov.*

```
int age, i, total = 0, ages[32];
for(i = 0; i <= 32; i++) {
    age = receive();
    if(age == -1)
        break;
    ages[i] = age; // overflow when i=32
    total  += ages[i];
}
show(total);
```

Code 1: Single-byte overflow. `ages` has 32 elements, and the loop read/write 33 elements from/into the buffer.

dressSanitizer (ASan) [22] is the most widely used memory sanitizer. It achieves detection of (silent) memory errors by obtaining the variable layout information from the source code, which is unfortunately not portable to binary-only programs. Since the ability of ASan cannot be ported to binaries, binary-program-oriented sanitizers (e.g., Dr. Memory [23], QASan [24], ASan-Retrowrite [25]) were proposed. Owing to the agnosticism regarding buffer boundaries in binaries, they are not able use exact buffer boundaries for detection. Instead, they use the boundary of a stack frame and the allocation sizes in a heap as the approximate boundary of each buffer. This method works for most pointer-corruption overflows, but will inevitably fail to detect memory corruptions that only modify the adjacent variables of a buffer, as such modifications are within the approximate boundary. A very recent work, MTSan, by Chen et al. [26] adopted a novel progressive object recovery scheme to recover objects in binaries and uses a customized binary rewriting solution to instrument binaries with the memory-tagging-based memory safety sanitizing policy [26]. However, due to the challenge of inferring the boundary of a data object, MTSan exhibits a relatively low detection rate (75.83% on buffer overflow dataset), which means a portion of buffer overflows were not detected. Furthermore, it requires the hardware feature of the ARM processor and cannot be generalized to other architectures. Therefore, in this paper, we aims to propose a more generic approach to detect silent buffer overflows with a high accuracy.

Code 1 shows an example of silent buffer overflow incident. The incident only corrupts one adjacent variable (`total`) of the buffer (`ages`) in the stack frame, which results in incorrect calculated results. (We will explain this example in more detail in Section 3.) Because the overflow does not cross the boundary of the stack frame, existing binary-program-oriented sanitizers are not capable of detecting it. That means, checking approximate boundaries is not adequate for spotting silent buffer overflows. Therefore, we propose an approach that does *not* rely on boundary information. In particular, our approach is based on the following observation:

Key Observation. The "peer" elements inside a buffer have *similar* data dependencies and access patterns, whereas the buffer's adjacent variables demonstrate *different* data dependencies and access patterns, because they serve a different purpose in the program logic.

A silent buffer overflow detection principle can be derived from the key observation: *A buffer overflow happens when variables adjacent to a buffer are accessed in a different way from how they are accessed in normal cases. That is, data dependencies and access patterns are different.* We denote the phenomenon of temporarily different data dependencies coexisting at a single address as a *dataflow conflict* and the phenomenon of temporarily different access patterns as an *access conflict*.

Following this principle, we propose to combine heuristic rules and deep learning models, which are complementary to each other, to detect dataflow conflicts and access conflicts. Each conflict is defined by a contrast between the program's behavior during a buffer overflow incident and the behavior during normal memory accesses. We first use heuristic rules to handle cases where the conflicts can be explicitly specified. However, although rule-based methods usually generate zero false positives, a main limitation is that rules are hard to be generalized: A rule working perfectly for one type of memory layouts may easily fail for another type of memory layouts. Due to the diversity of memory layouts for a buffer in the real-world, it is not practical to manually generate the detection rules that applies to all memory layouts. Moreover, it's even more challenging to craft detection rules when some memory layouts involve complex data structures such as `structure` and `union`, in which it's difficult to identify the dataflow conflicts and access conflicts.

Therefore, for cases that cannot be handled by easily crafting heuristic rules, we design a deep neural network (DNN) to automatically learn the differences between the program's data dependencies and access patterns during a buffer overflow incident and those patterns during normal memory accesses. Specifically, we use a collection of silently overflowed buffers to train the DNN model. First, we design a novel graph structure to hold several important features obtained from program executions, including data dependencies and types of accesses for each address/element involved in either normal buffer accesses or buffer overflow incidents. Second, we adopt a TreeLSTM neural architecture to leverage the novel graph structure and learn the representations of the data dependencies and access patterns (for each address/element involved in either normal buffer accesses or buffer overflow incidents) from the sequential operations recorded in our execution traces. Please note that the set of elements involved in a buffer overflow incident cover the overflowed region. Third, for each buffer overflow incident, we conduct peer-to-peer comparisons and calculate the differences between the representations learned for each "peer" address/element involved in the incident. The difference calculations are directly inspired by the aforementioned Key Observation. For normal buffer accesses, we also calculate the differences which should indicate "no difference" or "little difference". Fourth, an LSTM model is trained to assess the calculated differences and predict whether a silent buffer overflow has happened. Finally, we integrate the trained DNN model into a fully automatic toolchain, *DeepSanitizer*.

In summary, we make the following main contributions:

- We design and implement *DeepSanitizer*, which is a sanitizer that can detect silent buffer overflows in binary-only programs. *DeepSanitizer* leverages a newly-identified detection principle and adopts a hybrid approach that combines traditional binary analysis and deep learning.

- To the best of our knowledge, this is the first work that can detect silent buffer overflows that only corrupt adjacent variables in binary-only programs. The core idea of our approach (i.e., *dataflow conflicts* and *access conflicts*) is different from all existing sanitizers.
- Our evaluation experiments on 5,000 data samples and 30 real-world CVEs show that *DeepSanitizer* can effectively detect silent buffer overflows in binary programs.

2 Background

2.1 Memory Corruption and Memory Sanitizer

Memory Corruption Vulnerabilities represent one of the oldest challenges in computer security, particularly affecting programs written in unsafe programming languages like C or C++. The absence of memory safety (or type safety) in these languages results in vulnerabilities that allow attackers to alter the program's behavior or potentially gain full control over the control-flow. Over decades, buffer overflow has persisted as a prominent memory corruption vulnerability plaguing the cyberspace, due to the prevalence of buffers in programs and the relative simplicity of exploiting such vulnerability.

While a seemingly straightforward solution would be to avoid using these languages and rewrite vulnerable programs in type-safe languages. Unfortunately, this is impractical not only due to the large amount of existing C/C++ code running on various platforms but also because of the necessity of low-level features required for performance-critical programs, such as operating systems.

Memory sanitizers are dynamic tools designed to detect memory safety violations (e.g., memory corruption) in programs developed in unsafe languages [27, 28]. These tools monitor program behaviors and verify whether memory accesses violate spatial safety or temporal safety. Spatial safety dictates that a memory access should stay within the bounds of the intended object, while temporal safety ensures that only valid objects can be accessed. Any violation of these safety constraints would lead to the program aborting and triggering an error message.

Most memory sanitizers rely on static analysis to identify memory layout and program instrumentation to monitor program's runtime behaviors. When the source code of a target program is available, memory sanitizers typically insert checks during the compilation phase. For instance, ASan utilizes an LLVM compiler pass to establish shadow memory to record the variable layout and embed *Redzones* between variables in the shadow memory. The *Redzone* acts as a guard region to provide a safety margin around allocated memory, making it easier to identify and diagnose memory-related issues during runtime. It also incorporates runtime libraries that validate memory access during program execution by querying the shadow memory [22]. Out-of-bound access (such as buffer overflow) falls in the *Redzone* and triggers an error.

In cases where the source code is not accessible, memory sanitizers resort to dynamic binary translation or static binary instrumentation to inject checks [23, 29] for each memory access. However, substantial information is lost during the compilation phase, including type information, language-specific syntax, memory layouts. This loss makes it nearly impossible to comprehensively sanitize a program and eliminate all spatial memory safety violations using binary instrumentation [22, 27].

2.2 Representation Learning for Binary

In the literature, several different kinds of deep learning models have been adopted to learn representation for binary programs. The neural network design and features engineering are problem-specific and require the domain knowledge of human experts. Based on how features are encoded before feeding into a deep learning model, existing works can be broken down into three categories.

Category I (*Learn from Raw Bytes*) process raw bytes in machine code, without considering the semantics of assembly code [30, 31, 32]. Each of the 256 different bytes are usually represented as a vector (e.g, one-hot), respectively. Then the vectors for each raw byte in an instruction are aggregated to represent the instruction.

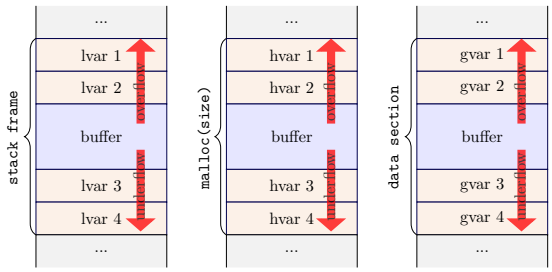
Category II (*Learn from Static Assembly Code*) first dissembles the binary instructions to assembly language using a disassembler and then extracts features via static assembly code analysis [33, 34]. Works in this category usually have their models to learn (encodings) from the extracted features. Both categories, despite the differences in their strategy of processing binary instructions, mainly focus on capturing lower-level semantics (e.g., opcode-operand correlations, instruction-instruction adjacency, other lexical semantics) of a binary program.

Category III (*Learn from High-level Representation*) first extracts high-level semantics, such as control flow graph, data flow graph, and abstract syntactic tree, from the binary program and then adopt graph neural network and its variants to learn representation for the program [35, 36, 37, 38]. One noticeable challenge to learn the high-level semantic from the program is to effectively capture the dependency among the variables and code. Accordingly, graph attention mechanism [39], GraphLSTM [40], GGNN [41], and TreeLSTM [42] can potentially deal with long-range dependencies, and have different strengths and limitations.

In our work, we adopt the TreeLSTM [42] to learn data dependency patterns and access patterns for buffer access. Because compared with general graph neural networks [43], TreeLSTM has demonstrated better capability in capturing long-range dependencies in a graph [44]. Furthermore, its lightweight model size alleviates the need for an excessively large dataset.

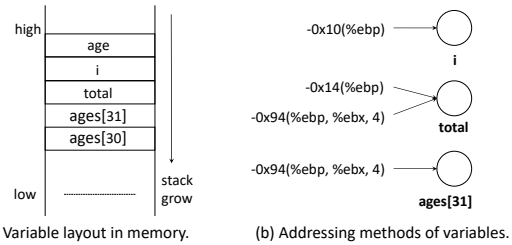
3 Motivating Example

Code 1 uses a loop to store `age` into an array (`ages`), and store their summation into `total`. The array `ages` has 32 elements in total: `ages[0]` to `ages[31]`. However, the programmer made a mistake when writing the loop control variable, resulting in a widely seen bug, i.e., 1-byte overflow. It is very easy to confirm this vulnerability when the source code is available. That is, the buffer overflow corrupts one adjacent variable, i.e., `total`, when `i` reaches 32. However, because this corruption does not affect the program's control flow, this program would not crash - it could continue the execution to the end with an incorrect output.



(a) On the Stack. (b) On the Heap. (c) On the Data Section.

Fig. 1: Silent BOF in three types of memory regions.



(a) Variable layout in memory. (b) Addressing methods of variables.

Fig. 2: Memory layout and variable access pattern around overflowed region.

```
1   sub     $0x94,%esp                ; AT&T syntax
2   movl    $0x0,-0x10(%ebp)          ; i = 0
3   movl    $0x0,-0x14(%ebp)          ; total = 0
4   jmp     target2
5 target1:
6   call    <receive>
7   mov     %eax,-0xc(%ebp)           ; age = receive()
8   cmpl    $0xffffffff,-0xc(%ebp)    ; if (age == -1)
9   je      target3                   ; break
10  mov     -0x10(%ebp),%ebx          ; [i]
11  mov     %eax,-0x94(%ebp,%ebx,4)   ; ages[i] = age
12  add     %eax,-0x14(%ebp)          ; total += ages[i]
13  addl    $0x1,-0x10(%ebp)          ; i++
14 target2:
15  cmpl    $0x20,-0x10(%ebp)         ; i <= 32
16  jle     target1
17 target3:
18  mov    -0x14(%ebp), %edi          ; show(total)
19  call   <show>
```

Code 2: The x86 assemble code compiled from Code 1.

The above example is not the only situation in which a silent buffer overflow happens. In fact, several situations can result in silent buffer overflow: a) A silent buffer overflow could happen on the stack frame if it does not corrupt any code/data pointers, it only corrupts the adjacent variable(s), and it does not cross the boundary of a stack frame, b) A silent buffer overflow could happen on the heap under the same set of conditions (on the stack) except that it does not cross the boundary of the allocated chunk, c) A silent buffer overflow could happen in the data section of a program under the same set of conditions except that it does not cross the boundary of the data section (shown in Figure 1).

Code 2 is the x86 assembly code of Code 1. Combining with the source code in Code 1, we can map the assembly code to C code as shown in the comments on the right side. Since the assembly code uses `sub -0x94, %ebp` to allocate space for the local variables in the function and all the remaining assembly code only access the allocated stack space, the boundary between variables is unclear.

From the assembly code, we can only observe that a sequence of consecutive addresses are accessed 33 times, and it is not clear whether there is a buffer overflow. This case reflects the difficulty of detecting a buffer overflow on the stack when the source code is **not available**. Existing work that tries to recover data objects from binary suffers from a low accuracy, mainly due to the challenge of recovering the boundary of data objects [26].

If we zoom in on the memory layout of the stack (Figure 2.a) by combining Code 1 and Code 2, we can observe that the location of variable `total` on the stack is accessed through two types of memory addressing (Figure 2.b), i.e., `-0x14(%ebp)` and `-0x94(%ebp,%ebx,4)`, respectively. The first type of addressing is the regular access to `total`, while the second type of addressing is through pointer dereference with an offset, i.e., `ages[32]`.

Hence, the different types of memory addressing to the same address can enable us to identify the potential overflow point. We denote this kind of pattern as *addressing type conflict*, which is one type ($\mathcal{AC}1$) of *access conflict*. Besides this type of access conflict, we have also identified three additional types of access conflicts as shown in Table 1. When an overflow against a char array corrupts an adjacent 4-byte integer variable, the overflowed region would be accessed with both 4-byte write and 1-byte write, which we denote as *size conflict* ($\mathcal{AC}3$). $\mathcal{AC}2$ happens when a region adjacent to a buffer is accessed through two different pointers, while $\mathcal{AC}4$ happens when the region is accessed through different levels of pointer dereference. We will illustrate $\mathcal{AC}2$ - $\mathcal{AC}4$ shortly in Section 6.

Data dependency is a fundamental type of relationships between statements and predicate expressions (or instructions) in a program. Data dependency relationships result in data flows from one variable to another. For example, in Code 1, since there is a data flow from `age` to `ages`, we say that `ages` depends upon `age`. Similarly, `total` depends upon `ages`. In general, "peer" elements in a buffer (e.g., `ages`) usually have a similar kind of – if not the same kind of – data dependency relationships between an element in the buffer and the other variables (excluding the buffer) in the program. In contrast, the kind of data dependency relationships between one adjacent variable of the buffer and the other variables are usually quite different. Accordingly, we have identified two types of *dataflow conflicts*, denoting the different kinds of data dependency relationships that coexist in the overflowed region. As shown in Table 1, $\mathcal{DC}1$ indicates the presence of two distinct structures of data dependency graph in an overflowed region, while $\mathcal{DC}2$ denotes a varied set of operations performed along the dependency path.

In the rest of this paper, to facilitate the discussion of access conflicts and dataflow conflicts, we name the pattern of how a variable is read and written as *access pattern* and the specific data dependency graph structure as *data dependency pattern*.

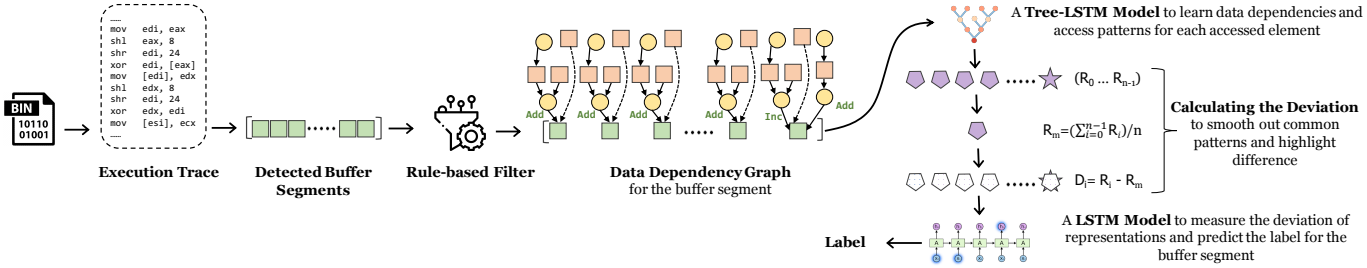


Fig. 3: Approach overview.

TABLE 1: Access conflicts and Dataflow conflicts.

Confl.	Acc. to Buf.	Acc. to Adj. Var.	Confl. Type
$\mathcal{AC}1$	poi dereference	direct access	addressing type confl.
$\mathcal{AC}2$	deref. w/ poi. P_1	deref. w/ poi. P_2	pointer confl.
$\mathcal{AC}3$	write 1 byte	write 4 bytes	size confl.
$\mathcal{AC}4$	double dereference	single dereference	dereference confl.
Confl.	**DF of Buffer**	**DF of Adj. Var.**	**Confl. Type**
$\mathcal{DC}1$	structure1	structure2	flow confl.
$\mathcal{DC}2$	logical op.	arithmetic op.	operation confl.

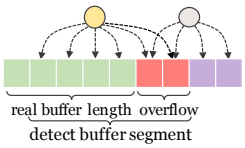


Fig. 4: A example of detected buffer segment.

4 Problem Scope and Our Approach

4.1 Scope of Research

Silent buffer overflow (SBOF) is a type of buffer overflow that only corrupts one or several adjacent variables and does not cause crashes, faults, or exceptions when it happens. This work aims to detect silent buffer overflows in binary programs. Similarly to ASan, this work only focuses on silent buffer overflow caused through sequential buffer access, and random buffer access is out of the scope. Based on the statement from [45], buffer overflow caused through random buffer access can only be detected with bound-check, which requires accurate buffer length information that is only available when the source code is available.

4.2 Workflow of Our Approach

The key idea behind *DeepSanitizer* is to exploit *access conflicts* and *dataflow conflicts* to automatically detect silent buffer overflow in binary programs through dynamic analysis. It is natural to conclude a method of crafting and applying detection rules from these conflicts. However, due to the diversity of memory layouts for a buffer in the real world, it is not practical to manually generate these detection rules. We opt for a hybrid approach that combines traditional data flow analysis with deep learning methods.

Figure 3 presents the workflow of our approach starting from a binary program as an input. As shown on the left of Figure 3, *DeepSanitizer* begins with the execution of the program, generating an **execution trace** containing the sequence of executed instructions and accessed addresses (Please note that the research of seed generation to trigger buffer overflow in a binary program is orthogonal to this work and thus out of the scope of this paper). Then, because buffer overflow happens around a buffer, *DeepSanitizer* identifies actual **buffer segments**, which is a sequence of memory regions that are accessed through a set of sequential memory read/write (Section 5). Such buffer segment can include all or partial original buffer and even adjacent variables when a buffer overflow happens, as shown in Figure 4. Therefore, we utilize the term **element** to denote an accessed region during a single read or write operation of a buffer segment. As an instruction may read one or more contiguous bytes of memory, we use the start address to denote an element.

Since the number of buffer segments in a binary program is usually far greater than the number of buffers that can be overflowed, a set of heuristic rules are employed in **the rule-based filter** to filter the above-identified buffer segments and keep only buffers that can potentially be overflowed (Section 6). Our observation during this process indicates that over 50% of the above-identified buffer segments is free of overflow. However, it is hard to only use heuristic rules to confirm the remaining cases due to the intricate data dependency pattern and access pattern. Therefore, we adopt a deep learning method to draw the classification boundary for the remaining cases.

After confirming the range of the potential buffer segments, *DeepSanitizer* performs data flow analysis on each element (including the original elements and may also include the adjacent variables) in this buffer segment and constructs each element's **Buffer Data Dependence Graph** (BDDG) (as shown in the middle of the Figure 3) based on the analysis result (Section 7). Each BDDG contains the *data dependency pattern* and *access pattern* of the specific element, which is later learned by a **TreeLSTM model** to generate a representation ($\mathbf{R}_i$) for each element (shown on the right of Figure 3). Subsequently, the **deviation** ($\mathbf{D_i}$) of each element's representation in the buffer segment is calculated. This deviation serves to filter out common patterns and emphasize differences, representing *access conflicts* and *dataflow conflicts*. Finally, an **LSTM model** is employed to learn the sequence of deviations and predict the occurrence of a buffer overflow (Section 8).

5 Buffer Segment Detector

Our objective is to identify silent buffer overflows during program execution by examining *access conflicts* and *dataflow conflicts*. However, verifying each memory access could result in an unacceptable overhead. Consequently, we must

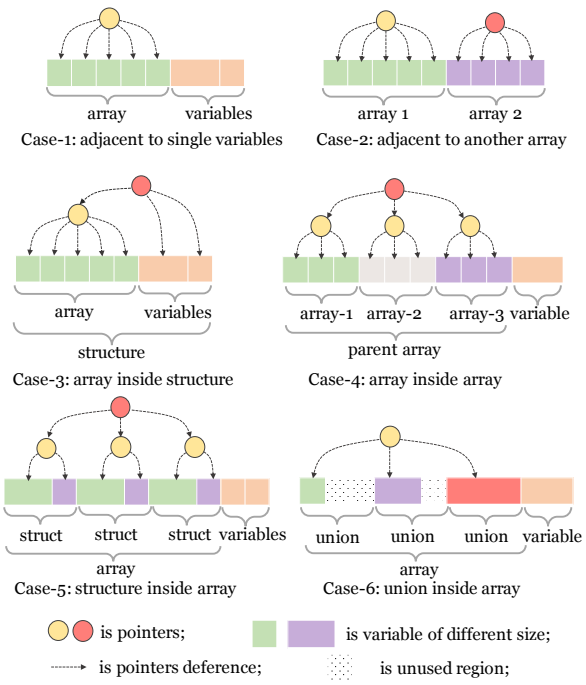


Fig. 5: Different types of memory layout around a buffer.

only target at memory accesses to buffer segments. Given our focus on buffer overflow within the context of sequential buffer access, the following distinctive features derived from sequential buffer access are used for this purpose.

1) *Element*s are accessed sequentially, either from low addresses to high addresses or vice versa, with a fixed stride.
2) Accessing the elements uses the same root pointer.

Taking buffers on the stack as an example, the following steps can be used to detect buffer segments:

1) First, the <start, end> address and the <allocation, destroy> point of a stack frame can be identified. We use this information to track if variables are live at certain points.
2) Second, each memory access can be associated with a memory address.
3) A chunk of accesses can be viewed in one buffer segment if a) it uses the same root pointer for these accesses, b) the memory addresses to be accessed have a fixed stride, c) the access ordered by its address value is the same as its execution order or the reverse of the execution order, and d) all accesses are inside a pair of <start, end> and <allocation, destroy>.

It is important to note that this approach cannot precisely pinpoint the actual start and end points of the actual buffer. It detects a set of sequential access to a sequence of memory regions (i.e., buffer segment).

6 Rule-based Filter

Section 3 explains that although the array length is lost in the binary program, it is still possible to detect silent buffer overflow. The insight behind this method is that the pattern of accessing the same variable differs when a buffer overflow happens. As a result, it is natural to conclude a strategy to detect silent buffer overflow in binary programs, i.e., summarizing and utilizing similar insights for different buffer cases. For example, based on the categories of adjacent areas, Figure 5 demonstrates six types of memory layout for a buffer.

TABLE 2: Different types of conflicts for different types of memory layout.

Cases	Buffer Layout	Potential Conflicts
Case-1	adjacent other single variables	$\mathcal{AC}1$, $\mathcal{AC}3$, $\mathcal{DC}1$, $\mathcal{DC}2$
Case-2	adjacent to another buffer	$\mathcal{AC}2$, $\mathcal{DC}1$, $\mathcal{DC}2$
Case-3	arry inside a struct	$\mathcal{AC}3$, $\mathcal{AC}4$, $\mathcal{DC}1$, $\mathcal{DC}2$
Case-4	array inside array	$\mathcal{AC}1$, $\mathcal{AC}3$, $\mathcal{AC}4$, $\mathcal{DC}1$, $\mathcal{DC}2$
Case-5	struct array	$\mathcal{AC}4$, $\mathcal{DC}1$, $\mathcal{DC}2$
Case-6	union array	$\mathcal{AC}1$, $\mathcal{AC}4$, $\mathcal{DC}1$, $\mathcal{DC}2$

Different types of memory layouts for a buffer can result in different types of access/dataflow conflicts, and therefore, demanding different rules to detect buffer overflows. Table 2 summarizes different types of conflicts for each type in Figure 5.

Case-1 shows a memory layout that a buffer is adjacent to standalone variables. A buffer overflow can result in *addressing type conflict* and *size conflict* in this case. That is, the following rule can be applied to detect silent buffer overflow for this case: A buffer overflow happens if any elements in an array area accessed through different accessed sizes (e.g., 1 byte vs. 4 bytes) or different addressing types (e.g., pointer dereference vs. direct access).

This detection rule cannot work for *Case-2*, in which the elements in two adjacent arrays have the same size. *Addressing type conflict* and *size conflict* cannot be observed when a buffer overflow happens in the first array. Therefore, the detection rule for *Case 2* is: A buffer overflow happens if any element in an array is accessed through different pointers.

In *Case-3*, the array is inside a structure. When a buffer overflow happens, the region adjacent to this array is still accessed through the same pointer, i.e., this structure's pointer. However, because accessing the elements in the array is through two levels of pointer dereference (i.e., struct_poi→array_poi→element) while accessing other fields in this structure is through one level of pointer dereference (i.e., struct_poi→element), *dereference depth conflict* can be observed. When the adjacent field has a different size from the element in the array, *size conflict* can also be observed.

Case-4 shows a two-dimensional array, i.e., a parent array contains three child arrays. If a buffer overflow happens in the third child array, the adjacent variable is corrupted, and *addressing type conflict* and *size conflict* can be used to detect the overflow. If a buffer overflow happens in the first child array, although the same root pointer was used in the first-level dereferences (i.e., poi→array1_poi and poi→array2_poi), the second-level dereferences use different pointers (i.e., array1_poi→element and array2_poi→element). Such slight differences could be used to detect the overflow.

When endeavoring to generalize detection rules, we face two challenges: First, the real-world presents various types of potential memory layouts, demanding substantial

manual efforts to formulate rules for each of them. Second, crafting rules for specific memory layouts proves to be exceedingly challenging. For example, in *Case-5* and *Case-6*, the array's element type is `structure` and `union` respectively, capable of accommodating different data types in one same element. Therefore, different elements in the same array can have different access/dataflow patterns. Developing detection rules for these cases requires accounting for each valid access pattern, necessitating increased manual efforts. In summary, the existence of various memory layouts and the combination of different access/dataflow patterns can intricately complicate the analysis of access/dataflow conflicts, presenting significant challenges in crafting accurate and general detection rules.

On the flip side, formulating heuristic rules to exclude certain benign cases is more attainable than striving for the creation of detection rules that guarantee completeness. Despite the potential for false positives, this approach significantly diminishes the number of buffers necessitating further analysis. Therefore, we employ a set of heuristic rules to remove buffer segments that are free of overflow. Due to the page limit, the following shows only two rules and both rules can individually remove a set of buffer segments.

1) If all accesses to a buffer segment consistently utilize the same data pointer at every level of dereference, the buffer segment does not have an overflow.
2) If all elements in a buffer segment have the same data dependency graph of the same structure, the buffer segment does not have an overflow.

These rules can triage the buffer segments into those with potential overflows and those potentially free of overflow, and thus help narrow down the number of buffer segments to be further examined in the majority of binary programs. In the following sections, for these buffer segments with potential overflows sifted by the rule-based filtering, we call them as **candidate buffers**. We further apply deep neural network to identify silent buffer overflow among these candidate buffers.

7 Buffer Data Dependence Graph

Central to training the neural network model is to feed in essential information that is necessary for detecting silent buffer overflow in binary programs. Drawing from *access conflicts* and *dataflow conflicts* outlined in Table 1, we utilize the following features to train the model.

1) The address order of elements in the buffer segment.
2) The memory-accessed size and addressing type.
3) Each element's data dependency.
4) The opcode of one instruction in the execution trace.

Definition 1 (Customized Data Dependence Graph)**.** *In a Customized Data Dependence Graph $CDDG(N, E)$, each node in N represents a live variable or an instruction of the program, and each edge in E represents a data dependency relation between two nodes in N. An instruction node n_i is directly data-dependent on another variable node n_v if and only if n_v is the source operand of n_i and is used to calculate the value of the destination operand of n_i, or n_v is directly copied to the destination operand. A variable node n_v is directly data-dependent on an instruction node n_i if n_v is the destination operand of the n_i. A variable node n_{v1} is indirectly data-dependent on another variable node n_{v2} through an instruction node n_i if n_{v2} is used to address n_{v1} in n_i.*

Definition 2 (Buffer Data Dependence Graph)**.** *A Buffer Data Dependence Graph (BDDG) is a subgraph of a Customized Data Dependence Graph (CDDG) which is sliced to only include all nodes data-dependent on or being data-dependent on by the elements in the buffer segment, and the edges these nodes are connected to.*

In particular, BDDG has two types of nodes and three types of edges, and the selected features are *embedded* either as node or edge features. Two types of nodes in BDDG are:

- a *o-node* represents an operation performed on a variable and the node feature is the operation's opcode.
- a *v-node* represents a *live* variable and the node feature includes the variable size.

Three types of edges in BDDG represent different kinds of dependencies, and they are:

- a *d-edge* describes a direct data dependency (i.e., variable assignment).
- an *i-edge* represents an indirect dependency (i.e., pointer dereference).
- an extra type of edge (*a-edge*) connects a *o-node* performing direct memory access to its accessed variable.

In practice, we find that a deep learning model can not effectively learn access patterns and data dependency patterns from one single graph which comprises all types of dependency together. Consequently, we split the Buffer Data Dependency Graph into three distinct subgraphs, adopt three models of the same architecture, and let each model specifically focus on one type of graph:

1) Define-Flow Graph (DeFG) represents the data dependency involved in writing the elements of the buffer.
2) A Use-Flow Graph (UFG) represents the data dependency associated with reading the elements of the buffer.
3) A Pointer-Flow Graph (PFG) represents the data flow engaged in calculating the pointer to address the buffer.

Such "divide and conquer" learning strategies improve the effectiveness of learning different types of dependencies from the BDDG.

Given the above definitions, Figure 6 illustrates the Define-Flow Graph (DeFG), Use-Flow Graph (UFG), and Pointer-Flow Graph (PFG) for the buffer overflow case in Code 1. The symbol $\prec$ is employed to signify the order (based on memory address) of elements in the buffer segment. The DeFG visually shows that each element in the buffer segment is data-dependent on the return value of `receive`() function. The UFG illustrates that the variable `total` is data-dependent on each element's value. The PFG shows the calculation of the pointer for accessing each element in the buffer segment. From Figure 6, it is clear that three graphs of the overflowed region (inside red dashed lines) deviate significantly from the graphs for other elements in the buffer segment.

8 Neural Network Design

We design a three-phase deep neural network to ascertain whether a buffer may be overflowed: *Phase 1* is to learn

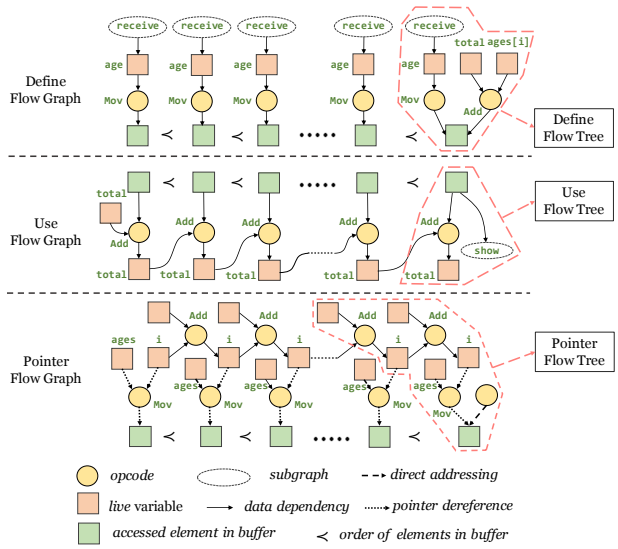


Fig. 6: Buffer Data Dependence Graph.

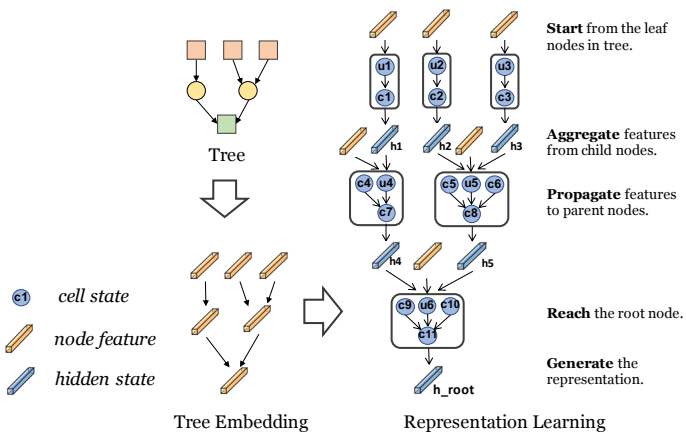


Fig. 7: Representation learning for data dependency tree through TreeLSTM.

the access pattern and data dependency pattern for each element in the buffer segment; *phase 2* is dedicated to comparing and contrasting the learned representations of data dependency and access; *phase 3* is to measure the conflicts and then predict a label for the buffer segment.

Representation Learning[1] for Access. This phase acquires knowledge of *access patterns* and *data dependency patterns* from the BDDG. Graph Neural Network [46, 47] and Tree-LSTM [44] stand out as two widely adopted approaches for learning data dependency from graph representations of programs. Existing literature shows that Tree-LSTM excels in learning long-range data dependencies in graphs, extending up to 15 hops [44]. This feature is extremely helpful to learn *access patterns* and *data dependency patterns*. Therefore, we opt for Tree-LSTM to effectively learn the representation of each element in the buffer.

However, the Tree-LSTM can only learn data in the shape of tree. Therefore, we first need to convert three graphs into tree structure. Starting from the *buffer node* (representing an element in the buffer) in the graph x, we generate a Define-Flow Tree (DeFT) by backward slicing the Define-Flow Graph (DeFG). Because nodes in the define-flow are in the upstream from the buffer node in the graph, we adopt a backward slicing which initiates the process from buffer node and traverses all the path in the opposite direction of the edges in the graph. Similarly, a Pointer-Flow Tree is also generated by backward slicing the Pointer-Flow Graph (PFG). In addition, the buffer node in the use-flow is in the downstream of the graph. We generate Use-Flow Tree (UFT) by forward slicing the Use-Flow Graph (UFG), which traverses all the path along the direction of the edges in the graph.

Second, the node and edge features in the generated trees are derived from its corresponding graphs and embedded into feature vectors. Then, we leverage a Tree-LSTM to learn the representation for each tree. Figure 7 illustrates the learning process of TreeLSTM on the DeFT from Figure 6. As shown in the figure, the representation learning starts from the leaf node in the tree. Following the direction of edges, it constantly learns and aggregates features from leaf nodes, and propagates to the parent nodes. Finally, when the learning process reaches the root node, it generates the representation for the tree, which encloses the graph structure, and node/edge features. The learning process on the UFT and PFT are similar.

Third, for a buffer segment with n addresses being accessed, we generate three ordered sequences of representation: $(\mathbf{R}_i^d)\, 0 \leq i \leq n-1$, $(\mathbf{R}_i^u)\, 0 \leq i \leq n-1$, and $(\mathbf{R}_i^p)\, 0 \leq i \leq n-1$, corresponding to DeFG, UFG, and PFG, respectively.

Finally, after TreeLSTM models learn the representations for the DeFT, UFT, and PFT for each buffer node in the buffer segment, we need to merge these three types of feature so that we can compute deviations that capture all features in next step. Technically, we amalgamate the three sequences of representation into a unified sequence $((\mathbf{R}_i)\, 0 \leq i \leq n-1)$ through element-wise concatenation:

$$\mathbf{R}_i = concatenate(\mathbf{R}_i^d, \mathbf{R}_i^u, \mathbf{R}_i^p) \tag{1}$$

Deviation Calculation and Overflow Prediction. The learned representation encapsulates the *data dependency pattern* and *access pattern* of a buffer segment in one sequence. To emphasize the distinctions between individual buffer accesses, suppress common patterns, and enable the detection of conflicts, we compute deviation representations $((\mathbf{D}i)\, 0 \leq i \leq n-1)$ for the buffer segment through the following calculation (shown in Figure 3):

$$\begin{aligned} \mathbf{R}_{\mathrm{m}} &= \left(\sum_{i=0}^{n-1} \mathbf{R}_i \right) / \mathrm{n} \\ \mathbf{D}_{\mathrm{i}} &= \mathbf{R}_i - \mathbf{R}_{\mathrm{m}}, 0 \leq i \leq n-1 \end{aligned} \tag{2}$$

where $\mathbf{R}_i$ is results from Equation 1, $\mathbf{R}_m$ is the mean of all the representation in a buffer segment. The computation generates a sequence of deviations: $(\mathbf{D}_i)\, 0 \leq i \leq n-1$, which emphasize the distinctions in *access patterns* and *data dependent patterns* between the accesses inside the buffer and outside the buffer, and eliminating commonalities.

1. Representation learning is paradigm in machine learning where the algorithm learns to automatically discover and create meaningful representations or features from raw data.

In the last step, we adopt a sequence model (i.e., LSTM) to measure the sequence of deviation, and then predict the label for the buffer segment. Technically, since "peer" elements inside a buffer have similar data dependencies and access patterns, the values within $\mathbf{D}_i$ tend to be low, whereas higher values are anticipated within buffer segments that contain overflows. For LSTM, it is easy to detect such difference from the sequence $((\mathbf{D}_i)\, 0 \leq i \leq n-1)$.

9 Implementation

DeepSanitizer is implemented on a 64-bit Linux system with the Intel x86 Instruction Set Architecture. It consists of four main components: 1) A binary program tracer based on `Intel Pin` [48], which collects various runtime information, including executed instructions, accessed memory addresses, and values. It is implemented in 2,277 lines of C++ code. 2) A buffer segment detector designed to identify sequential buffer access in the execution trace, which is implemented in 2,223 lines of C++ code. 3) A trace analyzer responsible for constructing the BDDG for each detected buffer segment, which is implemented in 7,062 lines of C++ code. 4) A deep learning model that predicts whether a buffer overflow happens on this buffer, which leverages Pytorch and comprises 956 lines of Python code.

10 Experiments and Evaluation

10.1 Dataset Generation

Training a neural network model requires a substantial amount of data, typically thousands of data samples. However, buffer overflow vulnerabilities are limited in public databases such as Common Vulnerabilities and Exposures (CVE). This limitation is particularly strong when our focus is on programs written in C/C++. We have explored the utilization of the Juliet Test Suite for constructing our training set. However, our investigation revealed that only a limited portion of the test suite pertains to silent buffer overflow. Furthermore, the diversity of the memory layout within the Juliet Test Suite is not sufficiently comprehensive to encompass the various cases elucidated in Figure 5. In response to this challenge, our decision is to generate a dataset (including 80% and 20% as training and testing set, respectively) to train and test the model and utilize vulnerabilities from real-world vulnerability databases to evaluate the model's performance on real-world cases.

The basic idea of generating one data sample starts with inserting buffer overflow vulnerabilities into the source code of real-world programs. Three strategies are employed to perform this insertion: 1) Reducing the length of buffer. 2) Relaxing the bound-check for buffer access – allowing the access to reach and surpass the buffer boundary. 3) If there is no direct bound check for a buffer, we directly increase the value of the pointer for this buffer.

It is time-consuming to manually insert vulnerabilities. Therefore, we employ an LLVM pass to automatically identify buffers and buffer accesses in the program, and applying these strategies to modify the source code. As a result, the binary program compiled from the modified source code has the buffer overflow vulnerabilities inside. Additionally, different inputs are used to generate the execution traces for the program, which are later utilized to construct BDDGs (data sample).

In practice, we found that more than half of the inserted buffer overflow can result in program crash. We simply exclude this type of buffer overflow because they can be detected by the existing binary sanitizer already. As a result, all overflow samples in our dataset are *silent* buffer overflow, that cannot be detected by existing tools.

After one data sample is generated, it requires a label (to indicate overflow or not) for the purpose of model training. Once again, manually labelling the data sample is not practical and an automatic way is required. Hence, before compiling vulnerable programs, another LLVM pass is developed to collect source code information (e.g., buffer length), and insert checkpoints to verify whether a buffer access exceeds the boundary. Finally, each data sample (i.e., BDDG) is labelled with the information whether there is a silent buffer overflow recorded in this graph.

We try to increase the diversity of the buffer layouts when selecting the real-world programs. We find that in principle the Case 6 in Figure 5 exists, however it is hardly to find this case in the real-world programs. As a result, we use 5 types of buffer layout out of 6 cases discussed in the Figure 5. Finally, we select 10 real-world programs (`Binutils`, `Coreutils`, `GCC`, `Curl`, `Bloaty`, `OpenSSL`, `Python`, `PHP`, `Sqlites`, `LibPNG`) in different categories and they are widely used in related researches. `Binutils` and `Coreutils` are two essential software packages commonly found in Unix-like operating systems. `GCC` is a popular compiler. `Curl` is a command-line tool for making HTTP requests and interacting with various network protocols. `Bloaty` is a size profiler for binaries. `Python` and `PHP` are two program language interpreters. `Sqlites` is a database program. `OpenSSL` and `LibPNG` are general-purpose libraries.

In the end, 5,000 data samples including 2030 samples with overflowed buffers and 2970 samples without overflow are generated. It is worth noting that the source code is primarily used for generating training data. Once the model is trained, it becomes capable of detecting silent buffer overflow in binary-only programs.

10.2 Model Performance

Experiment Setup. To train our model, we tune a few hyper-parameters including different numbers of layers, size of hidden states, and dropout rates to find the best configuration. We adopt 5-fold cross-validation to avoid any random noises and get stable results. In each round, we randomly select 80%, 20% of data samples as a training set and a testing set. The programs included in the testing set are not included in the training set. That is, we always test our model on *previously unseen programs*.

Model Performance. The first row in Table 3 shows the metrics *Accuracy*, *Precision*, *Recall*, and *F1* on the testing set. On average, our model achieves an accuracy of 98.80% and an F1 score of 98.71% on the testing set, which is 98.71% detection rate and 1.12% false positive rate. The detection rate of DeepSanitizer is better than the related work (MTSan [26]), which adopts data object recovery to

TABLE 3: Comparison between our model and other baseline model.

Model	Acc	Precision	Recall	F1
RNN	0.4977	0.4994	0.5557	0.5260
LSTM	0.4948	0.4929	0.5136	0.5030
RGCN	0.8411	0.8699	0.8158	0.8175
TreeLSTM + LSTM	0.9374	0.8873	0.9531	0.9190
Our Design	**0.9880**	**0.9870**	**0.9871**	**0.9871**

detect silent buffer overflow and achieves a detection rate of 79.83%.

Comparison with Baseline Models. To assess the efficacy of our model, we conduct a comparative analysis with potential alternative designs. Our comparative investigation centers around addressing the following key questions:

1) Can other widely used models in binary program analysis, such as RNNs and LSTMs, address the issue by analyzing the instruction sequence?
2) Is it possible for a Graph Neural Network (GNN) model to directly identify silent buffer overflow on BDDG?
3) How effective is the Equation 2 (deviation calculation) in our model design?

First of all, RNN and its variants, such as LSTM, have been widely employed in prior binary analysis [30, 31, 32, 49]. Our baseline selects instructions from the execution trace utilized for buffer access and schedules them into an instruction sequence. Subsequently, RNN and LSTM models are applied to directly learn the instruction sequence, predicting the occurrence of a silent buffer overflow.

The outcomes presented in Table 3 reveal that RNN and LSTM models fail to recognize any patterns associated with silent buffer overflow, which indicates that the challenge cannot be effectively addressed by a model based on RNN/LSTM, even when it is trained with instruction sequences.

Second, a Graph Neural Network (GNN) is a natural choice for learning features from graphs. For instance, Allamanis *et al*. construct dependency graphs and adopt Gated Graph Neural Networks [41] to learn representations for the entire program [47]. Therefore, we design an experiment to adopt Relational Graph Convolutional Network (RGCN [47]) to directly predict whether a node in BDDG is produced when the buffer overflow happens. RGCN is selected from many GNN designs because it is able to learn graphs with multiple types of nodes and edges. The results in Table 3 indicate that while RGCN demonstrates certain ability of learning access patterns from BDDG, but it is not as effective as our model. The primary reason for this inefficacy lies in the GNN model's inability to capture long-range data dependencies, as demonstrated by the Tree-LSTM [44].

Third, we did an *ablation study* [2] to understand the importance of the deviation computation step between the Tree-LSTM and LSTM model. In our neural network design, we firstly calculate the deviation (Equation 2) of the learned tree representations from the Tree-LSTM, and then feed the deviation to the LSTM model to measure the sequence of division and predict the final label for the buffer. To affirm this design choice, we conducted comparison experiments. Specifically, we directly use the learned representations from the TreeLSTM model as the input for the LSTM model, instead of using the deviations of these representations as the input. Table 3 shows this model's performance, which has a decrease when compared with our model. This outcome substantiates our assertion that the deviation metric effectively emphasizes the distinctions among buffer access patterns and data dependency patterns, and thus helps improve the performance of silent BOF detection.

2. An ablation study investigates the performance of an AI system by removing certain components to understand the contribution of the component to the overall system.

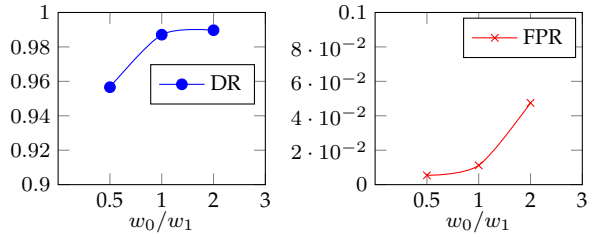


Fig. 8: Trade-off between DR and FPR through tuning the weighted loss function.

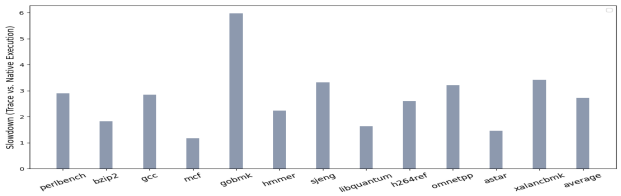


Fig. 9: Runtime overhead of *DeepSanitizer*.

Trade-off the DR and FPR. In real-world scenarios, certain use cases necessitate a high detection rate, while others prioritize a low false positive rate. To address this objective, we adjust the penalties for false positives or false negatives during the training phase using a weighted (i.e., w_0/w_1) loss function defined as:

$$Weithted_Loss = w_0 \cdot Loss_{class0} + w_1 \cdot Loss_{class1} \quad (3)$$

where, $Loss_{class0}$ is the loss associated with predictions of class 0 (normal buffer); $Loss_{class1}$ is the loss associated with predictions of class 1 (overflow buffer); w_0 and w_1 are weights assigned to the losses of class 0 and class 1, respectively. A larger w_0 and w_1 increase the penalty of false prediction of negative samples and positive samples respectively. Therefore, a lower value of w_0/w_1 aids in achieving a lower False Positive Rate (FPR) at the expense of a lower Detection Rate (DR), whereas a higher value of w_0/w_1 facilitates a high DR at the cost of a higher FPR.

Figure 8 visually represents the trade-offs between detection rate and false positive rate for various w_0/w_1 values. Specifically, the model attains a detection rate as high as 98.97% (with a FPR of 4.75%) when choosing $w_0/w_1 = 2$ and an FPR as low as 0.54% (with a DR of 95.66%) when choosing $w_0/w_1 = 0.5$.

Runtime Overhead. The implementation of *DeepSanitizer* consists of 4 components, among which only the tracer slows down the execution of the target program, and other components are performed offline, through other threads.

TABLE 4: The list of real-world vulnerability and detection results (in column of $\mathcal{D}$).

Vulnerability Information		*Results*	*Vulnerability Information*		*Results*
ID	Name	$\mathcal{D}^1$	ID	Name	$\mathcal{D}$
2004-0597	pngslap	✓	2004-2093	rsync	✓
2004-1120	proz	✓	2004-1299	vilistextum	✓
2004-1255	2fax	✓	2005-0101	newspost	✓
2004-1257	abc2mtex	✓	2005-3862	unalz	✓
2004-1261	asp2php	✓	2005-4807	as-new	✓
2004-1262	bsb2ppm	✓	2007-1465	dproxy	✓
2004-1275	html2hdml	✓	2007-2683	mutt	✓
2004-1278	jcabc2ps	✓	2009-1759	ctorrent	✓
2004-1279	jpegtoavi	✓	2009-2286	compface	✓
2004-1287	nasm	✓	2010-2891	smisubtree	✓
2004-1288	o3read	✓	2014-8322	aireplay	✓
2004-1289	pcal	✓	EDB-890	psnup	✓
2004-1290	pgn2web	✓	EDB-9264	stftp	✓
2004-1292	ringtonetools	✓	EDB-14904	fcrackzip	✓
2004-1297	unrtf	✓	EDB-15062	rarcrack	✓

Therefore, we evaluate the impact of our tool on the execution of the analyzed program.

The industry-standard SPEC CPU®2006 benchmark suite includes SPECint® [50] benchmark suite. Specifically, we run each program in the SPECint® twice: a native execution that runs with default configuration, and a traced execution that is attached through our Intel Pin based tracer. Figure 9 summarizes the runtime overhead of the tracer in *DeepSanitizer*. The result shows that *DeepSanitizer* incurs 2.718x slowdown on average. Compare to other related works, we argue this is acceptable overhead for vulnerability discovery and analysis.

10.3 Detecting Real World Buffer Overflows

Our model is trained on a generated dataset, prompting the need to assess whether the generated dataset accurately represents the distribution of real-world vulnerabilities. Consequently, we utilize the trained model to identify vulnerabilities within the Common Vulnerabilities and Exposures Database (CVE) and Exploit Database (EDB) [51].

Specifically, we utilize a list of real-world buffer overflow vulnerabilities studied in prior literature [52, 53]. These vulnerabilities come from CVE database and proved to be reproducible. For each buffer overflow vulnerability in the list, we attempt to compile the corresponding target programs in our experimental environment (i.e., Linux with x86 Instruction Set), excluding those that cannot be successfully compiled. Second, we reuse the input provided by existing study [53] to trigger each vulnerability. Third, we exclude the overflows that result in crash and only keep the silent buffer overflows. Finally, we generate a list of 30 programs with silent buffer overflows. The "ID" and "Name" columns in Table 4 presents the vulnerability ID and program name, respectively. Please note that selection of the vulnerabilities follows the order of the list [52]. To ensure the selected samples reflect the distribution of real-world vulnerabilities, we did not intentionally select or exclude any cases.

DeepSanitizer is able to successfully detect all 30 silent buffer overflows, and pinpoint the location of the vulnerabilities. It spots the overflow location in a program by mapping the buffer segment to its location in the execution trace.

To investigate why our model achieve such a good detection rate on the real-world vulnerabilities, we analyze in depth the buffer types and memory layout of the vulnerable buffers. We observe that the variety of real-world buffers are much less than the variety of our training set. First, all the types of vulnerable buffer in 30 cases are of `char`, or `int`. Second, the 30 real-world cases contains three types of the most popular memory layouts: in CVE-2004-1289, the overflowed array `token` is in-between two other arrays: `move` and `temp`. In CVE-2004-1261, the overflowed array `temp` is adjacent to another array, i.e., `token`. In CVE-2004-1279, the overflowed buffer `name` is inside a structure `Jpeg_Data`. In other words, our training set is comprehensive, which covers sufficient memory layouts and buffer types, leading to the high detection rate in the real-world cases.

10.4 Analyzing the FPs and FNs

To study the limitations of our method, we analyze the BDDGs associated with False Positives (FPs) and False Negatives (FNs) identified by the model predictions. Our study reveals that most FNs occur either because the corrupted variables were not utilized during the executions or the overflowed region is unused memory. For instance, one variable `rels` in `readelf` is a heap buffer. However, during the runtime, there are no *live* variables adjacent to it in the heap. As a result, an overflow of this buffer would not expose any *access conflicts*.

The reasons for False Positives (FPs) are heterogeneous. Some FPs arise from excessively complex dataflows within the buffer, while others result from multiple types of access patterns to the buffer. For instance, taking the array `symtab` in `readelf` program as an example, this array is an array of `elf_internal_sym`, which is a structure comprising seven domains and four types of data. In addition, the function `print_dynamic_symbol()` reads elements from this array, and in different branches, the elements inside the array are accessed in distinct ways. We posit that the presence of diverse dataflow patterns and access patterns within the buffer is the primary cause of false positives.

10.5 Comparing with Other Sanitizers

Table 5 shows the type of buffer overflows that can be detected by *DeepSanitizer* and other binary-level sanitizers on the columns of "Stack BOF", "Heap BOF", and "Global BOF". As mentioned in the Section 3, most existing binary sanitizers can only detect stack buffer overflow that crosses the boundary of stack frames (CSF), and heap buffer overflow that crosses the allocated region (CAR) and is not capable of detecting No-CSF, No-CAR, and Global BOFs, which are three main categories of silent BOF.

Our method can detect all silent buffer overflows that corrupt adjacent variable(s). As shown in the Table 5, the only case that cannot be detected by our method is the CAR, which represents the case that a buffer occupies a whole allocated heap chunk. In this case, the buffer may not be adjacent to other variables in the heap, and the overflow will not corrupt any variable. Hence, *DeepSanitizer* fails to identify heap buffer overflows when the buffer occupies an entire allocated memory region or is located at the end of an allocated region.

TABLE 5: **Comparison between our tool with other existing binary level sanitizers.**

Santizers	Stack BOF		Heap BOF		Global BOF (GBOF)	Detection Rate	Just-in Time	Runtime Overhead
	Cross Stack Frame	No-CSF	Cross Allocated Region (CAR)	No-CAR				
ASan-Retrowrite	✓	✗	✓	✗	✗	–	✓	2.57x
QASan	✓	✗	✓	✗	✗	–	✓	35.5x
Dr. Memory	✓	✗	✓	✗	✗	–	✓	12.0x
MTSan	✓	✓	✓	✗	✓	75.83%	✓	1.82x
DeepSanitizer	✓	✓	✗	✓	✓	98.71%	✗	2.718x

MTSan [26], the state-of-art binary level sanitizer, is also capable of detecting silent BoF such as (No-CSF, global BOFs) by inferring the boundary of data objects and leveraging the hardware features on ARM processor. However, the achieved detection rate (75.83%) is relevantly lower than our method. Besides, it assigned the same tag for dynamically allocated memory region on the heap, therefore cannot detect No-CAR overflow which only corrupt the variables inside the allocated region.

Furthermore, existing binary-level sanitizers detect buffer overflows through runtime bound checks, which can detect overflow before it happens. Our method needs the complete buffer data flow graph, which is only available after the target program finishes its execution. Hence, *DeepSanitizer* cannot achieve Just-in-Time detection of the silent buffer overflow (second to last column in Table 5). As a result, our tool cannot be deployed as an online detection mechanism to detect attacks exploiting buffer overflow vulnerabilities.

The final column assesses the runtime overhead. MTSan demonstrates the most favorable runtime overhead (1.82x) by capitalizing on the ARM processor's hardware feature – Memory Tagging Extension (MTE) in aarch64 – to tag allocated memory regions and their corresponding pointers. However, its reliance on this hardware feature also limits its applicability to a broader range of Instruction Set Architectures.

11 Discussion and Future Work

This paper shows the capability of *DeepSanitizer* to effectively detect and locate silent buffer overflow vulnerabilities. We envision two potential applications in real-world scenarios.

First, in the context of developing integrated systems, developers widely incorporate third-party binary code, such as libraries. Silent overflows may occur due to bugs in the binary code or inappropriate usage by developers, leading to logic errors. Our tool aids in detecting and locating silent buffer overflows, providing developers with insights into the root causes of logic errors and facilitating bug fixes. Second, as demonstrated in Section 10.3, *DeepSanitizer*, if applied to check each execution of the AFL, can assist in identifying real-world silent buffer overflows in binary-only programs.

However, our approach does have some limitations. First of all, due to the absence of buffer length at the binary level, *DeepSanitizer* may not achieve a 100% detection rate. Additionally, *DeepSanitizer* relies on pointer analysis to identify sequential buffer accesses, making it unable to detect out-of-bound access during random buffer access. Furthermore, there are evasion attacks that can bypass our detection by manipulating the pattern of buffer access. We identify two potential attacks: First, if an attacker attains permission to modify the code (e.g., shell code injection attack [54]), it can easily change the access pattern by manipulating instructions that are used to access the vulnerable buffer and then bypass our detection. Fortunately, the widely deployed defenses [5, 55] against control flow hijacking attacks can effectively prevent an attacker from modifying the code. Second, in code reuse attacks (e.g., ROP attack [56]), an attacker can selectively execute the code in the program space resulting in different buffer access patterns. However, because ROP attacks often exploit a buffer overflow vulnerability to overwrite the return address with their ROP chain, the stack overwriting trait of ROP attacks can still be detected by widely deployed defense mechanism such as CFI [5].

Future Works. *DeepSanitizer* specializes in detecting silent buffer overflows, which fall into the category of spatial errors. It would be valuable to explore the potential of identifying temporal errors (e.g., Use-After-Free) in binary-only programs.

The current prototype is implemented on the Linux System with the x86 Instruction Set. Its effectiveness in other operation system with different instruction sets can also be investigated. Furthermore, compared with other binary-level sanitizers, *DeepSanitizer* has a slightly higher runtime overhead than existing tools. In future works, we could try to reduce the runtime overhead in the following way: incorporating hardware features (e.g., ARM ETM [57]) and selectively tracing the code that is only related to buffer access.

12 Conclusion

In this work, we design and implement *DeepSanitizer*, a memory sanitizer that addresses the main limitations of existing binary-level memory sanitizers. Based on the observation that the data flow and memory access patterns are abnormal in the overflowed region, we propose to use *access conflicts* and *dataflow conflicts* to represent the observed anomaly. *DeepSanitizer* combines a set of heuristic rules based on traditional program analysis and a specially trained deep neural network to detect the access conflicts and dataflow conflicts caused by silent buffer overflows. The evaluation experiments against our newly generated dataset show that *DeepSanitizer* achieves a high detection accuracy of 98.8%. We have also conducted experiments to detect a collection of real-world silent buffer overflows. The results show that although the existing binary-level memory sanitizers cannot detect these silent buffer overflows, *DeepSanitizer* successfully detected all of them.

References

[1] L. Szekeres, M. Payer, T. Wei, and D. Song, "Sok: Eternal war in memory," in *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, pp. 48–62.

[2] M. Abadi, M. Budiu, U. Erlingsson, and J. Ligatti, "Control-flow Integrity," in *Proceedings of the 12th ACM Conference on Computer and Communications Security*, 2005.

[3] B. Niu and G. Tan, "Per-Input Control-Flow Integrity," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015.

[4] M. Zhang and R. Sekar, "Control Flow Integrity for COTS Binaries," in *Proceedings of the 22nd USENIX Security Symposium*, 2013.

[5] V. van der Veen, D. Andriesse, E. Göktaş, B. Gras, L. Sambuc, A. Slowinska, H. Bos, and C. Giuffrida, "Practical Context-Sensitive CFI," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015.

[6] H. Hu, C. Qian, C. Yagemann, S. P. H. Chung, W. R. Harris, T. Kim, and W. Lee, "Enforcing Unique Code Target Property for Control-Flow Integrity," in *Proceedings of the 25th ACM Conference on Computer and Communications Security (CCS)*, Toronto, ON, Canada, Oct. 2018.

[7] M. Corporation, "Control Flow Guard," https://msdn.microsoft.com/en-us/library/windows/desktop/mt637065(v=vs.85).aspx, 2016.

[8] Clang Project, "Clang Control Flow Integrity," https://clang.llvm.org/docs/ControlFlowIntegrity.html, visited on August 6, 2021.

[9] Android, "Android Control Flow Integrity," https://source.android.com/devices/tech/debug/cfi, visited on August 6, 2021.

[10] Chromium Project, "Chromium Control Flow Integrity," https://www.chromium.org/developers/testing/control-flow-integrity, visited on August 6, 2021.

[11] H. Shacham, "The Geometry of Innocent Flesh on the Bone: Return-into-libc Without Function Calls (on the x86)," in *CCS*, 2007.

[12] T. Bletsch, X. Jiang, V. W. Freeh, and Z. Liang, "Jump-Oriented Programming: A New Class of Code-reuse Attack," in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security*, 2011.

[13] F. Schuster, T. Tendyck, C. Liebchen, L. Davi, A.-R. Sadeghi, and T. Holz, "Counterfeit Object-oriented Programming: On the Difficulty of Preventing Code Reuse Attacks in C++ Applications," in *Proceedings of the 36th IEEE Symposium on Security and Privacy*, 2015.

[14] Nergal, "The Advanced Return-into-lib(c) Exploits," http://phrack.com/issues.html?issue=67&id=8.

[15] S. Chen, J. Xu, E. C. Sezer, P. Gauriar, and R. K. Iyer, "Non-Control-Data Attacks Are Realistic Threats," in *Proceedings of the 14th USENIX Security Symposium*, 2005.

[16] H. Hu, S. Shinde, S. Adrian, Z. L. Chua, P. Saxena, and Z. Liang, "Data-Oriented Programming: On the Expressiveness of Non-control Data Attacks," in *Proceedings of the 37th IEEE Symposium on Security and Privacy*, 2016.

[17] K. K. Ispoglou, B. AlBassam, T. Jaeger, and M. Payer, "Block oriented programming: Automating data-only attacks," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1868–1882.

[18] J. Pewny, P. Koppe, and T. Holz, "Steroids for doped applications: A compiler for automated data-oriented programming," in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2019, pp. 111–126.

[19] L. Cheng, H. Liljestrand, M. S. Ahmed, T. Nyman, T. Jaeger, N. Asokan, and D. Yao, "Exploitation techniques and defenses for data-oriented attacks," in *2019 IEEE Cybersecurity Development (SecDev)*. IEEE, 2019, pp. 114–128.

[20] H. Ye, S. Liu, Z. Zhang, and H. Hu, "{VIPER}: Spotting {Syscall-Guard} variables for {Data-Only} attacks," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 1397–1414.

[21] S. Ahmed, H. Liljestrand, H. Jamjoom, M. Hicks, N. Asokan, and D. D. Yao, "Not all data are created equal: Data and pointer prioritization for scalable protection against {Data-Oriented} attacks," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 1433–1450.

[22] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "{AddressSanitizer}: A fast address sanity checker," in *2012 USENIX annual technical conference (USENIX ATC 12)*, 2012, pp. 309–318.

[23] D. Bruening and Q. Zhao, "Practical memory checking with dr. memory," in *International Symposium on Code Generation and Optimization (CGO 2011)*. IEEE, 2011, pp. 213–223.

[24] A. Fioraldi, D. C. D'Elia, and L. Querzoni, "Fuzzing binaries for memory safety errors with qasan," in *2020 IEEE Secure Development (SecDev)*. IEEE, 2020, pp. 23–30.

[25] S. Dinesh, N. Burow, D. Xu, and M. Payer, "Retrowrite: Statically instrumenting cots binaries for fuzzing and sanitization," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 1497–1511.

[26] X. Chen, Y. Shi, Z. Jiang, Y. Li, R. Wang, H. Duan, H. Wang, and C. Zhang, "MTSan: A feasible and practical memory sanitizer for fuzzing COTS binaries," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 841–858. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/chen-xingman

[27] D. Song, J. Lettner, P. Rajasekaran, Y. Na, S. Volckaert, P. Larsen, and M. Franz, "Sok: Sanitizing for security," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 1275–1295.

[28] F. Gorter, E. Barberis, R. Isemann, E. Van Der Kouwe, C. Giuffrida, and H. Bos, "{FloatZone}: Accelerating memory error detection using the floating point unit," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 805–822.

[29] N. Nethercote and J. Seward, "Valgrind: a framework for heavyweight dynamic binary instrumentation,"

ACM Sigplan notices, vol. 42, no. 6, pp. 89–100, 2007.

[30] E. C. R. Shin, D. Song, and R. Moazzezi, "Recognizing functions in binaries with neural networks," in *24th {USENIX} Security Symposium ({USENIX} Security 15)*, 2015, pp. 611–626.

[31] Z. L. Chua, S. Shen, P. Saxena, and Z. Liang, "Neural Nets Can Learn Function Type Signatures from Binaries," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017, pp. 99–116.

[32] W. Guo, D. Mu, X. Xing, M. Du, and D. Song, "{DEEPVSA}: Facilitating Value-set Analysis with Deep Learning for Postmortem Program Analysis," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1787–1804.

[33] S. H. Ding, B. C. Fung, and P. Charland, "Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 472–489.

[34] X. Li, Y. Qu, and H. Yin, "Palmtree: Learning an assembly language model for instruction embedding," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 3236–3251.

[35] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, "Neural network-based graph embedding for cross-platform binary code similarity detection," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 363–376.

[36] J. Yan, G. Yan, and D. Jin, "Classifying malware represented as control flow graphs using deep graph convolutional neural network," in *2019 49th annual IEEE/IFIP international conference on dependable systems and networks (DSN)*. IEEE, 2019, pp. 52–63.

[37] H. Xue, S. Sun, G. Venkataramani, and T. Lan, "Machine learning-based analysis of program binaries: A comprehensive study," *IEEE Access*, vol. 7, pp. 65 889–65 912, 2019.

[38] W. Wang, G. Li, B. Ma, X. Xia, and Z. Jin, "Detecting code clones with graph neural network and flow-augmented abstract syntax tree," in *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2020, pp. 261–271.

[39] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.

[40] X. Liang, X. Shen, J. Feng, L. Lin, and S. Yan, "Semantic object parsing with graph lstm," in *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14*. Springer, 2016, pp. 125–143.

[41] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel, "Gated graph sequence neural networks," *arXiv preprint arXiv:1511.05493*, 2015.

[42] K. S. Tai, R. Socher, and C. D. Manning, "Improved semantic representations from tree-structured long short-term memory networks," *arXiv preprint arXiv:1503.00075*, 2015.

[43] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, 2020.

[44] Z. Wang, H. Wang, H. Hu, and P. Liu, "Identifying non-control security-critical data in program binaries with a deep neural model," *arXiv preprint arXiv:2108.12071*, 2021.

[45] P. Akritidis, M. Costa, M. Castro, and S. Hand, "Baggy bounds checking: An efficient and backwards-compatible defense against out-of-bounds errors." in *USENIX Security Symposium*, vol. 10, 2009, p. 96.

[46] A. Nair, A. Roy, and K. Meinke, "funcgnn: A graph neural network approach to program similarity," in *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2020, pp. 1–11.

[47] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to represent programs with graphs," *arXiv preprint arXiv:1711.00740*, 2017.

[48] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood, "Pin: building customized program analysis tools with dynamic instrumentation," *Acm sigplan notices*, vol. 40, no. 6, pp. 190–200, 2005.

[49] X. Li, Y. Qu, and H. Yin, "PalmTree: Learning an Assembly Language Model for Instruction Embedding," in *ACM CCS*, 2021.

[50] "Spec cpu 2006 int benchmark," Available: https://www.spec.org/cpu2006/CINT2006/, Standard Performance Evaluation Corporation, 2006.

[51] S. Özkan, "Cve details," *Retrieved*, vol. 16, p. 2017, 2017.

[52] Github, "Linuxflaw." [Online]. Available: https://github.com/mudongliang/LinuxFlaw

[53] D. Mu, A. Cuevas, L. Yang, H. Hu, X. Xing, B. Mao, and G. Wang, "Understanding the reproducibility of crowd-reported security vulnerabilities," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 919–936.

[54] K. Z. Snow, S. Krishnan, F. Monrose, and N. Provos, "{SHELLOS}: Enabling fast detection and forensic analysis of code injection attacks," in *20th USENIX Security Symposium (USENIX Security 11)*, 2011.

[55] N. Stojanovski, M. Gusev, D. Gligoroski, and S. J. Knapskog, "Bypassing data execution prevention on microsoftwindows xp sp2," in *The Second International Conference on Availability, Reliability and Security (ARES'07)*. IEEE, 2007, pp. 1222–1226.

[56] H. Shacham, "The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86)," in *Proceedings of the 14th ACM conference on Computer and communications security*, 2007, pp. 552–561.

[57] Z. Ning and F. Zhang, "Ninja: Towards transparent tracing and debugging on {ARM}," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017, pp. 33–49.